1st test solutions

So, my voice is still crap... the voice doctor says at least a few months, voice therapy etc, let's see.

1. We had to order the functions by asymptotic growth.
    1. $100n^2$
    2. n^{10} --- for the first two you had to understand that exponents dominate growth, and that a larger exponent (10 vs 1) even with a smaller leading constant (1 vs 100) will eventually dominate.

       (All of the other exercises could be solved by the transformation:

       $$a^b = 2^{b \log a}$$

       and by noticing that $2^{c_1} = O(2^{c_2})$ whenever $c_1 = O(c_2)$

       Hence we get, for example:
       $n^{\log n} = 2^{(\log n)^2}$, right? and $n^{10} = 2^{10 \log n}$ and since $10 \log n = O((\log n)^2)$ then the ordering becomes:
    3. $(\log n)^{\log n}$
    4. $n^{\log n}$
    5. $(n^2)^{\log n}$
    6. $(1.1)^n$
    7. $(100)^n$
    8. $\frac{n!}{2^n} \approx \frac{n^n}{2^n} \approx 2^{n \log n - n}$

Ex 2

This was not simple, but not difficult.

To solve this problem you had to make some drawings of undirected graphs and figure out when they had an Eulerian path.

Sadly and stupidly, the exercise asked for "Eulerian path", but I meant to have asked for "Eulerian cycle" which would make the exercise easy.

However the crucial observation was the same: After trying out a bit, you should have concluded that, if a path enters a vertex and the vertex still has edges that have not yet been used, I can then leave the vertex. This will happen exactly if the vertex has an EVEN number of edges! (I consume one edge when entering, but the number is even so I can always exit.)

So a graph G has an Eulerian cycle if and only if it is connected and every vertex has an even number of edges. This makes the condition easy to check.

To get an Eulerian path is a bit trickier. It turns out, a graph G has an Eulerian path if and only if it is connected and either:

1. every vertex has even degree (in this case the path is a cycle)
2. every vertex except for exactly two vertices has even degree.

Both conditions are easy to check.

Ex 3

Ok first, 0.5 points of Ex 3 were for the definition... did you know it?

$$A \leq_p B \iff \exists f \text{ polytime} \forall x \qquad x \in A \iff f(x) \in B$$

To prove that B is NP-complete, we need to prove that B is in NP, and that $X \leq_p B$ for some NP-hard problem X.

It was suggested that the reduction come from CLIQUE.

So we are given a graph G on n vertices and a number k and we need to output a graph G' on n' vertices such that

$$G \text{ has a clique of size } k \iff G' \text{ has a clique of size } n'/2$$

Understanding that this was what needed to be done (and saying it) was 0.5 points.

(G has a clique of size k iff it has a clique of size $\geq k$ because I can always take a subclique of a larger clique)

You had to figure out two tricks:

- If I take a graph G and add a vertex that connects to no other vertex, then the size of the largest clique of G doesn't change.
- If I take a graph G and add a vertex that is connected to all other vertices, then the size of the largest clique increases by 1.

So:

- If $k = n/2$, we can simply output $G' = G$.
- If $k > n/2$, i.e. $k = n - \ell$, for some $\ell < \frac{n}{2}$, we add v vertices without edges so that

  $$k = \frac{n+v}{2} \iff n - \ell = \frac{n+v}{2}$$

  i.e. we add $n - 2\ell$ vertices. The size of the maximum clique did not change, but the size of the graph increased to $n + v$ so that $k = \frac{n+v}{2}$.
- If $k < \frac{n}{2}$, we add $n - 2k$ vertices to G, all connected to all other vertices, so that $n' = 2n - 2k$ and G' now has a clique of size $\frac{n'}{2} = n - k$ if and only if G has a clique of size k.

  Indeed, if G has a clique of size k then that clique together with all the new vertices is a clique of size $k + n - 2k = n - k = \frac{n'}{2}$, and if G' has a clique of size $\frac{n'}{2} = n - k$ then at most $n - 2k$ of those vertices are new vertices, so at least k of those vertices already belong to G so they form a clique of G.
- To finish the proof (oops Luis) one should mention still that this is a polytime reduction.

Ex 4

One needed to prove that:

- DominatingSet is in NP
- VertexCover reduces to DominatingSet

Let's do part two.

We are given as input a graph G and a number k, and we wish to produce a graph G' and a number k' such that:

G has a vertex cover of size k iff G' has a dominating set of size k'.

Yes?

> So, I don't know how you guys study, but in my undergrad years I adopted the following method, which has served me well:
>
> - I will do enough exercises until I am confident that I can do any exercise of similar difficulty that is presented to me.
> - If I am not there yet, I solve more exercises:
>   - Find a new exercise
>   - Try to solve it
>   - Find a solution for the exercise somewhere (book, online, whatever)
>   - Compare
>   - find another exercise
>   - etc

Ok back to Ex 4.

The idea of the reduction was not super obvious (which is why I offered a hint.) The hint would have been the following:

The nodes of G' will be:

- For every vertex v of G I have a vertex v of G', and
- For every edge $\{u, v\}$ of G I have a ***vertex*** $w_{u,v}$ of G'.
- For every edge $\{u, v\}$ of G I put edges in G' between u, v and the extra vertex $w_{u,v}$.

Claim. G has vertex cover of size k iff G' has dominating set of size k

(this concludes the exercise)

Proof:

Let C be a vertex cover of size k. Then every edge is incident to a node of C. But then the same set C is a dominating set of G', because all vertices that were in G, plus the extra "edge vertices" are all neighbours or elements of C.

Conversely, let D be a minimal dominating set of G'.

The crucial observation here is that: if a special "edge vertex" w_{uv} is in D, I can replace it with vertex u or v, and the new set will still be a dominating set of the same size (I can't decrease it because D is minimal).

So I can assume without loss of generality that D only has the original nodes of G. Since every edge vertex w_{uv} is a neighbour of some node in D, it follows that D is also a vertex cover of G.

Exercise 5

We have to count and match parenthesis.

Crucial observation. If I keep a counter, and increase it whenever I see $($ and decrease it whenever I see $)$, then the set of parenthesis will be matched if and only if:

- throughout the counting process, I never see -1, and
- at the end the counter is 0.

We can do this by scanning the input from left to right and keeping the counter in the work tape. This takes $O(\log n)$ space.

Exercises 6,7, or 8

See the book. You had to remember the proof.